

Project Design for TriGuard: A Multimodal Explainable Content Moderation Pipeline

CM3070 Computer Science Final Project — CM3020 Artificial Intelligence
Template, Project Idea 1: Orchestrating AI Models to Achieve a Goal

Hamza — BSc Computer Science, University of London

June 2026

1. Project Overview

TriGuard is a prototype, locally deployable content-moderation pipeline that orchestrates three pre-trained perception models — one each for text, image and audio — with a local Large Language Model (LLM) acting as judge. For each piece of content the system produces a single structured decision: risk score, risk label, flagged modalities and written rationale. The goal is to demonstrate that a small, auditable, on-device pipeline can support human moderators on multimodal content that can be difficult for single-modality moderation tools to assess. TriGuard is decision-support, not autonomous moderation: every output is intended for human review.

2. Template Used

The project is built on the **CM3020 Artificial Intelligence** template, *Project Idea 1: Orchestrating AI Models to Achieve a Goal*. The template asks for the use of multiple pre-trained models combined toward a specific objective. TriGuard satisfies this by composing four established models (a text toxicity classifier, BLIP for image captioning, Whisper plus YAMNet for audio, and a local instruction-tuned LLM as judge) inside an explicit orchestration layer.

3. Domain and Users

TriGuard sits at the intersection of AI-assisted content moderation, trust and safety, multimodal machine learning, explainable AI and privacy-aware local AI. The intended users are: (i) **human moderators** who need structured evidence; (ii) **trust-and-safety teams** at small and medium platforms; (iii) **researchers and NGOs** studying online harm who need auditable tools; and (iv) **forum operators** with privacy-sensitive content who cannot send data to a third-party cloud. The system is an internal tool, not aimed at end users posting content. The regulatory context — the EU Digital Services Act and the UK Online Safety Act — makes transparency, auditability and risk assessment increasingly important (Gorwa, Binns and Katzenbach, 2020).

4. User Needs and Domain Requirements

Five user needs drive the design: **cross-modal coverage** (harm often crosses modalities (Kiela et al., 2020)); **clear explanations** that are defensible to users, regulators and appeals processes; **local deployment** so sensitive content is not sent to third-party APIs by default; **uncertainty reporting** when the system is unsure; and **manageable output** that a moderator can skim in seconds. The table below links each need to a design response and to how that response will be evaluated.

User / domain need	Design response	How it will be evaluated
Cross-modal coverage	Text, image and audio wrappers feed one orchestrator	Test unimodal and multimodal examples
Clear explanations	LLM judge produces a written rationale	Rationale usefulness and grounding review
Local deployment	Models run locally where possible	Record whether external API calls are avoided
Uncertainty reporting	Output includes uncertainties and review action	Check schema outputs and failure cases
Manageable output	Risk label, rationale and evidence shown in tiers	Peer feedback on readability of output

5. Design Rationale

TriGuard is designed as an **orchestration** system rather than a single new model. Three reasons drive this:

1. Strong pre-trained models already exist for each modality, making orchestration a practical design choice for this project (Lees et al., 2022; Radford et al., 2023; Li, J. et al., 2022).
2. Harmful content frequently has cross-modal signals that no single model can capture (Kiela et al., 2020).
3. A local LLM judge can read structured evidence, weigh it, and emit a *natural-language rationale* — addressing the explainability and governance requirements that classifier-only systems do not meet (Zheng et al., 2023; Gorwa, Binns and Katzenbach, 2020).

The system deliberately avoids training a new fusion model. Composition over training keeps the project tractable, preserves auditability of each model’s contribution, and allows individual models to be swapped without retraining.

6. System Architecture

The architecture (Figure 1) is a six-layer pipeline. Each layer has a clear contract. The implementation will define these contracts using Pydantic schemas so that model outputs can be validated consistently.

An **input handler** validates the payload. A **modality dispatcher** routes each provided field to the **text, image and audio wrappers**, each of which calls one pre-trained model and returns a normalised *evidence object*. The three evidence objects are collected into a **JudgeInput** and handed to the **local LLM judge** (Ollama running a small instruction-tuned model), whose output is constrained to a JSON schema. The judge returns a **TriGuardResult** containing the risk score, risk label, flagged modalities, rationale, uncertainties and latency, which is then displayed to the human moderator. A side branch sends the same result to the evaluation harness so every design decision is testable.

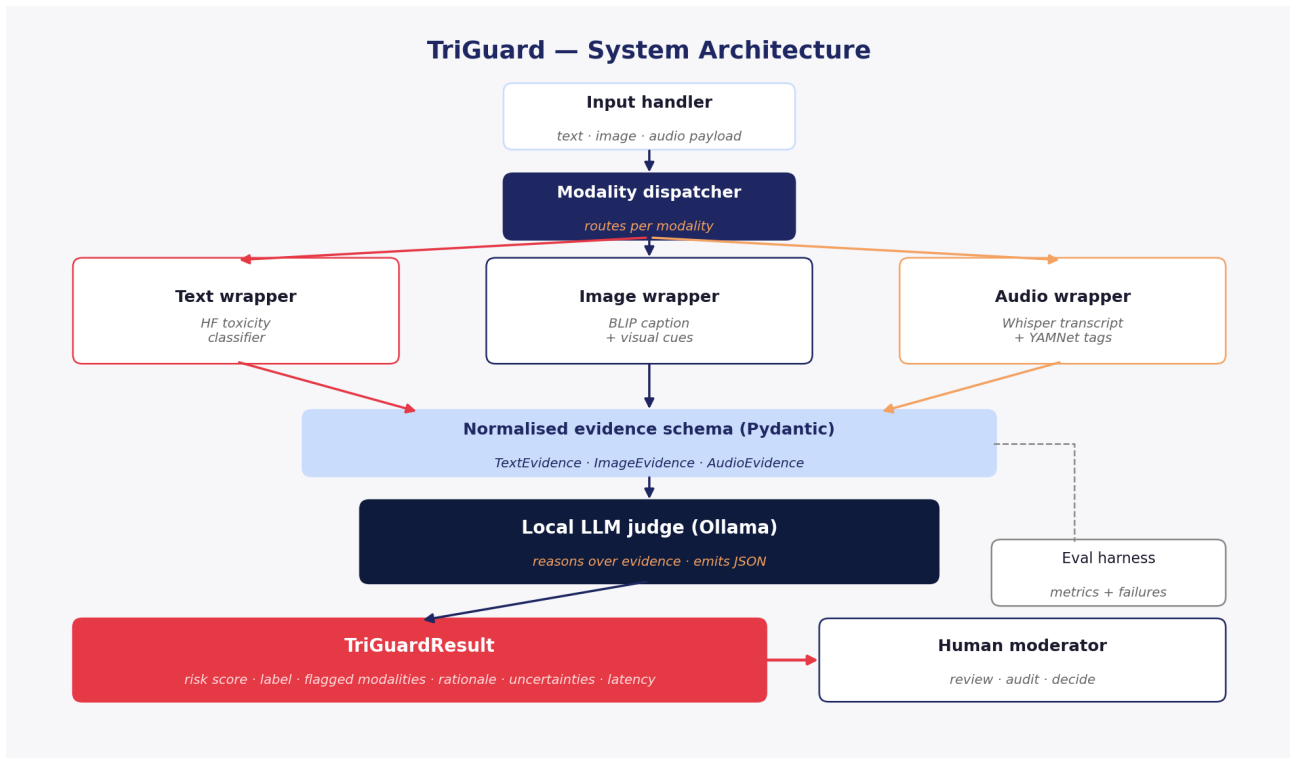


Figure 1: TriGuard system architecture.

7. Technologies and Methods

Component	Technology	Why
Language Schemas	Python 3.11 pydantic	Project standard; clean typing Strict validation of every model’s output
Text wrapper	Lightweight Hugging Face toxicity or sentiment classifier, selected during implementation based on availability, licence and model size	Provides a practical text-risk baseline without training a new model
Image wrapper	BLIP captioner (base)	Open captioner; output feeds the judge as text (Li, J. et al., 2022)
Audio wrapper	OpenAI Whisper (small) + YAMNet	Transcription and event tagging provide two different types of audio evidence (Radford et al., 2023; Gemmeke et al., 2017)
LLM judge	Llama-3-8B-Instruct via Ollama (Q4 quantised)	Runs locally on consumer hardware (Zheng et al., 2023, methodology)
Orchestration	Plain Python — no LangChain	Fewer moving parts; easier to test and explain

Component	Technology	Why
Demo	FastAPI	Minimal HTTP surface for reviewers
Testing	pytest + pytest-cov	Industry standard
Eval reporting	scikit-learn, pandas, matplotlib	Repeatable metrics; figures for the report

Model and package versions will be pinned during implementation to improve reproducibility.

8. Data Flow

A request enters as a typed payload with optional text, image and audio fields. The dispatcher calls only the wrappers needed; each returns a Pydantic evidence object. The orchestrator builds a JudgeInput, calls the LLM, and parses the response against JudgeOutput. If parsing fails, the orchestrator retries once with a stricter prompt; if it fails again, a borderline result is returned with `uncertainties=["judge_output_invalid"]` rather than crashing. The result is serialised as JSON with model versions and latency.

9. Prototype Plan

The prototype will be developed in two stages. The first stage will build a working end-to-end structure using mock wrappers for text, image and audio, so the data flow, schemas and orchestrator can be tested before heavy model integration begins. The second stage will replace the mocks one at a time with real models, starting with the text wrapper because it is the simplest to evaluate. This staged approach reduces risk and ensures a working prototype is available for the Preliminary Report even if some real model integrations take longer than expected.

- **Tier A.** Repository skeleton, schemas, mock wrappers, mock judge, orchestrator wiring, unit tests.
- **Tier B.** Replace mocks one wrapper at a time (text → image → audio → judge), each behind a feature flag, with tests at every step.

A short CLI (`python -m triguard.cli run sample.json`) and a FastAPI endpoint expose the pipeline so non-technical reviewers can drop in a sample and see the structured output.

10. Evaluation and Testing Strategy

Evaluation is designed alongside the system architecture so that each design choice can be tested. The prototype evaluation will focus on whether the pipeline works correctly, whether outputs follow the required schema, and whether the explanation is understandable. The final evaluation will compare unimodal and multimodal outputs where suitable, measure latency, and analyse failure cases. This avoids making claims about the system that are not supported by evidence.

Track	Question	Metric
T1 Schema	Outputs always valid?	% valid JSON
T2 Text	Text toxicity quality?	macro F1 on a public toxicity dataset, where access allows

Track	Question	Metric
T3 Image-text	Multimodal meme quality?	F1 / AUROC on Hateful Memes or a public fallback set
T4 Audio	Speech-to-text + event quality?	WER (Whisper); top-1 (YAMNet) on a small audio sample
T5 Orchestrator	Logic correctness	unit and integration tests; branch coverage
T6 Full pipeline	End-to-end behaviour	per-class F1; confusion matrix on a small hand-built set
T7 Rationale	Useful, grounded?	usefulness rating and grounding check
T8 Performance	Viable on consumer HW?	p50 / p95 latency; peak RAM; cold-start time

Every multimodal claim will be reported alongside a unimodal baseline, and no improvement will be claimed unless supported by evaluation results.

11. Work Plan and Gantt Chart

The plan below covers the full 24-week schedule. Each row lists the main tasks, the expected output, and a concrete contingency if the main path slips.

Weeks	Phase	Main tasks	Output	Contingency
1–2	Concept and pitch	Choose template; record pitch; set up repo	Pitch deck + video	Re-record if pacing off
3	Literature review	Select sources, build matrix, write review	Lit review PDF	Use 6 strongest sources if 8 too much
4	Project design	Architecture, technology, evaluation plan	Design PDF	Reduce feature scope if needed
5	Mock prototype	Schemas, mock wrappers, mock judge	Tier-A prototype	CLI only if FastAPI delays
6–9	Real wrappers	Text, image, audio models added gradually	Working wrappers	Keep mock fallback for hard models
10	Preliminary report	Combine literature, design, prototype, eval	Preliminary Report	Hybrid prototype if integration slips
11–17	Evaluation and demo	LLM judge, eval harness, FastAPI demo	Tier-B + results	Smaller dataset if runtime high

Weeks	Phase	Main tasks	Output	Contingency
18–24	Final report + submission	Draft report, revision, exam, final eval, video	Final report, code, video	Code freeze before exam revision

The plan totals around 230 hours across 24 weeks, with heavier weeks at 10, 18 and 24.

12. Contingency Plans

Risk	Trigger	Contingency
Local LLM too slow	Latency > 5 s per item on my laptop	Switch to a 3 B quantised model; cache calls
Hateful Memes access refused	Week 6 review	Use a public multimodal hate-speech dataset and a synthetic image-text set
Whisper too heavy	RAM above target	Use whisper-tiny; limit clips to 30 s
Judge JSON invalid	T1 valid rate < 100 %	Stricter prompt + Pydantic retry; else rule-based judge
Coding delayed	End of Week 9	Submit a mock-and-real hybrid rather than miss the deadline
Exam revision clash	Week 21	Hard coding stop at end of Week 20

A longer risk register will be maintained during implementation.

13. Inclusive and Ethical Design Considerations

Inclusive design is considered from the beginning of the project rather than added at the end. The output is designed to be understandable to non-technical reviewers, with a risk label, rationale and optional evidence view. Uncertainty is shown clearly so that doubtful cases are sent for review rather than automatically blocked. The demo interface will use readable text, clear labels and accessible contrast. The prototype is documented as English-focused, with multilingual support identified as future work. This approach follows the principle of born-accessible development, where accessibility is built into the design from the start rather than retro-fitted later (Lazar, 2023).

Ethically, this project involves potentially harmful content, so the design aims to reduce unnecessary exposure and avoid collecting private data. The prototype will use public datasets and small synthetic examples where appropriate. It will not scrape private social media content, collect participant posts, or store personal data. The pipeline runs locally so test content need not be sent to third-party APIs. The project is described as decision-support rather than autonomous moderation: the recommended action is advisory and should always be reviewed by a human moderator, to avoid overstating the reliability of the prototype.

14. References

- Gemmeke, J.F., Ellis, D.P.W., Freedman, D., Jansen, A., Lawrence, W., Moore, R.C., Plakal, M. and Ritter, M. (2017) ‘Audio Set: An ontology and human-labeled dataset for audio events’, *Proceedings of the 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. New Orleans: IEEE, pp. 776–780. <https://doi.org/10.1109/ICASSP.2017.7952261>
- Gorwa, R., Binns, R. and Katzenbach, C. (2020) ‘Algorithmic content moderation: Technical and political challenges in the automation of platform governance’, *Big Data & Society*, 7(1), pp. 1–15. <https://doi.org/10.1177/2053951719897945>
- Kiela, D., Firooz, H., Mohan, A., Goswami, V., Singh, A., Ringshia, P. and Testuggine, D. (2020) ‘The Hateful Memes Challenge: Detecting hate speech in multimodal memes’, in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. La Jolla: NeurIPS Foundation, pp. 2611–2624. arXiv:2005.04790.
- Lazar, J. (2023) ‘A framework for born-accessible development of software and digital content’, in *Human-Computer Interaction – INTERACT 2023*. Lecture Notes in Computer Science, vol. 14145. Cham: Springer, pp. 333–338.
- Lees, A., Tran, V.Q., Tay, Y., Sorensen, J., Gupta, J., Metzler, D. and Vasserman, L. (2022) ‘A new generation of Perspective API: Efficient multilingual character-level transformers’, *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’22)*. New York: ACM, pp. 3197–3207. <https://doi.org/10.1145/3534678.3539147>
- Li, J., Li, D., Xiong, C. and Hoi, S. (2022) ‘BLIP: Bootstrapping language–image pre-training for unified vision–language understanding and generation’, in *Proceedings of the 39th International Conference on Machine Learning (ICML 2022)*. PMLR vol. 162, pp. 12888–12900. arXiv:2201.12086.
- Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) ‘Robust speech recognition via large-scale weak supervision’, in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*. PMLR vol. 202, pp. 28492–28518. arXiv:2212.04356.
- Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E. and Stoica, I. (2023) ‘Judging LLM-as-a-judge with MT-Bench and Chatbot Arena’, in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. La Jolla: NeurIPS Foundation. arXiv:2306.05685.